Mitigating Retrieval Errors in RAG Pipelines Through Ambiguity Detection and Interactive Feedback

Mahmoud Ibrahim
School of Information Technology and Computer Science
Nile University
Cairo, Egypt
ma.mahmoud@nu.edu.eg

Dr. Walaa Medhat
School of Information Technology and Computer Science
Nile University
Cairo, Egypt
wmedhat@nu.edu.eg

2025 4th International Conference on Electronic Engineering (ICEEM) | 979-8-3315-8956-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICEEM66692.2025.11225289

***Abstract*—Retrieval-Augmented Generation (RAG) pipelines often struggle when multiple retrieved documents are semantically similar, resulting in ambiguous top-ranked outputs that degrade the quality of the generated responses. This ambiguity increases the risk of hallucinations, especially when the correct passage is not clearly distinguishable from other candidates. While existing solutions—such as linguistic preprocessing, score-based filtering, and retriever fine-tuning—have improved overall retrieval accuracy, they remain insufficient in addressing ambiguity caused by semantically close passages. In this work, we propose a novel interactive retrieval framework that detects ambiguous retrieval scenarios by analyzing the score distribution of top-ranked documents. When ambiguity is detected, the system selectively engages the user by presenting a list of candidate document titles, allowing them to choose the most relevant one. A dedicated classifier is trained to distinguish between confident and ambiguous cases, ensuring minimal disruption to the user experience. Experiments on an Arabic QA dataset demonstrate significant improvements in top-1 retrieval accuracy, validating the effectiveness of incorporating ambiguity detection and selective feedback into RAG pipelines.**

Keywords—Retrieval Augmented Generation (RAG); Ambiguous Results; Interactive Question Answering; Multilingual Retrieval; User-in-the-Loop; Ambiguity Classifier; Knowledge Graph.

I. Introduction

Question answering systems often rely on Retrieval-Augmented Generation (RAG) pipelines that retrieve relevant documents and then generate answers [1]. These systems can struggle, however, when the indexed documents are not semantically differentiated enough or the user's query is underspecified. Such issues can match multiple interpretations or topics, causing the retriever to return irrelevant or mixed results. The downstream answer generator may then produce incorrect answers based on the wrong context, undermining the system's reliability.

Ambiguous queries and ambiguous results are well-known challenges in information retrieval. Studies estimate that a significant fraction of users' queries are ambiguous [4]. Prior research has introduced metrics like the clarity score to quantify query ambiguity and shown that ambiguous queries correlate with poor retrieval performance [5].

A natural solution is to ask clarifying questions – a strategy explored in conversational search systems, where a single clarification question can yield a 170% improvement in precision@1 [3].

In open-domain QA, Min et al. found that over half of the questions were ambiguous, prompting the need for disambiguation by rewriting the query [2].

These works highlight that handling ambiguity is crucial for accurate QA.

In this paper, we propose an interactive ambiguity mitigation framework for RAG-based QA pipelines. Our approach automatically detects when the retrieved documents are likely ambiguous using a lightweight classifier that recognizes ambiguity from retrieval features (such as score margins, result diversity and entities overlap). Upon detecting ambiguity, the system suggests clarifying multiple-choice interpretations to the user with the suggested document titles that were proactively generated for each document via an LLM model. The user's selection is used to obtain the relevant document for answer generation.

We also constructed a new Arabic QA dataset (~10k Question-Context pairs) by combining and augmenting existing resources. Arabic was chosen as the focus language due to its high morphological richness and semantic ambiguity, which make it an ideal testbed for studying ambiguity detection in low-resource, linguistically complex settings [18] Handling ambiguity in Arabic also contributes to bridging the gap in multilingual QA systems, where Arabic is often underrepresented.

While our work focuses on Arabic and open-domain, we recognize that ambiguity occurs in many domains and languages, such as biomedical or legal domains, where documents may share overlapping terminology and structure. Extending our pipeline to these settings is a valuable direction for future research.

Using our dataset, we evaluate our framework on three dense retriever setups (baseline LaBSE [10], state-of-the-art e5-large embeddings [9], and a fine-tuned LaBSE model) to demonstrate the generality of improvements. To our knowledge, this is one of the first studies focusing on ambiguity resolution in Arabic open-domain QA.

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 21:55:19 UTC from IEEE Xplore. Restrictions apply.

Experimental results show substantial gains in Recall@1 (top-1 document recall) across all retrievers with the ambiguity mitigation turned on, with minimal interactions from the user only if needed.

The paper is organized as follows: Section II reviews related work on query ambiguity, user interaction for clarification, and dense retrieval models. Section III outlines our methodology, including dataset construction, knowledge graph creation, the retrieval models used, and the ambiguity classifier. Section IV presents experimental setup, evaluation metrics, and results. Finally, Section V concludes the paper and suggests directions for future work.

II. Literature Review

A. Ambiguity in Queries and QA

Ambiguous queries have long been recognized as problematic in information retrieval and QA systems. Cronen-Townsend and Croft [5] introduced the clarity score as a measure of query ambiguity (defined via Kullback–Leibler divergence between the query language model and the collection language model). A low clarity score indicates a vague query likely to retrieve diverse, mixed-topic results.

Similarly, Song et al. developed a taxonomy of query ambiguity. They used supervised learning to classify web queries as ambiguous or not, estimating that about 16% of real search queries are ambiguous [4].

These early works underpin our approach by confirming that automatic ambiguity identification is feasible with high precision and recall.

AmbigQA, proposed by Min et al. [2], highlighted that more than half of the questions in the Natural Questions dataset have multiple possible answers. Their solution was to generate all plausible answers and rewrite the question for each interpretation to resolve ambiguity. Our work shares the goal of handling multiple interpretations, but instead of offline question rewriting, we perform online disambiguation with user input on a single chosen interpretation.

Guo et al. introduced Abg-CoQA [6], a benchmark with nearly 1,000 ambiguous questions in dialogues. They defined several ambiguity types and demonstrated benchmarks for ambiguity detection, clarification question generation, and disambiguated answering. We draw on insights from these studies, especially in designing our ambiguity classifier and clarification strategy; however, our setting is single-turn QA (user asks one question) rather than a multi-turn conversation.

Recent studies have also proposed end-to-end LLM-based disambiguation mechanisms, such as using GPT-4 to resolve query intent via chain-of-thought reasoning [24], or neural models that generate clarification questions directly [25]. While effective in some settings, these methods can be resource-intensive and opaque. Our approach differs by incorporating a lightweight ambiguity classifier and leveraging user feedback as a fallback mechanism when confidence is low, a trade-off between automation and control.

B. Clarifying Questions and User Interaction

Prior research shows that inserting a clarification step can dramatically improve retrieval outcomes. Aliannejadi et al. formulated the task of asking clarifying questions in conversational search [3]. They found that even one well-chosen clarification question can yield over 170% improvement in precision at the top 1, highlighting the power of user-in-the-loop feedback.

Radlinski and Creswell (2017) and others at Microsoft Research have similarly advocated for search engines to engage users in a clarification dialogue when intent is uncertain [1]. Our approach applies this principle to QA, rather than directly returning the highest score document (and risking error), the system briefly asks the user to clarify.

C. Dense Retrieval Models

Dense retrieval has emerged as a powerful alternative to traditional sparse retrieval methods, especially in question answering and semantic search tasks.

These models map both queries and passages into dense vector representations using transformer-based encoders, allowing relevance to be computed via vector similarity in a shared embedding space. Notable examples include LaBSE (Language-agnostic BERT Sentence Embedding) [10], which was trained on parallel multilingual corpora to provide language-agnostic embeddings, and multilingual-e5 [9], which extends the E5 family of models to support a wide range of languages with instruction-tuned capabilities.

In the context of Arabic, models such as AraBERT [16] and MARBERT [18] have shown effectiveness in encoding syntactically rich and morphologically complex Arabic text. However, their utility in retrieval tasks often depends on domain adaptation and fine-tuning.

These dense retrievers serve as the foundation for modern retrieval-augmented generation (RAG) pipelines by enabling semantically relevant passage selection, even when lexical overlap is minimal.

III. Methodology

Our system augments a standard retrieval-augmented QA pipeline with an ambiguity detection by a clarification module.

Figure 1 illustrates the overall flow. When a user query comes in, we first retrieve a set of candidate documents using a dense retriever. We then extract features from this retrieval result to predict whether the results are likely ambiguous. If it is, we enter an interactive clarification loop with the user; otherwise, we proceed directly to answer extraction. Below, we describe the main components in detail.

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 21:55:19 UTC from IEEE Xplore. Restrictions apply.

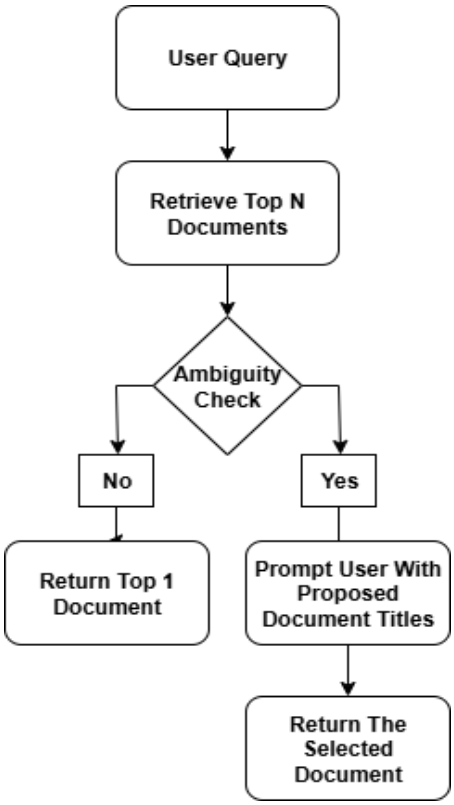

Fig. 1. The proposed RAG pipeline with ambiguity detection and user feedback. After retrieving and reranking top-N passages, the system checks for ambiguity. If confident, it returns the top result. If ambiguous, it prompts the user to select from a list of candidate titles. The final passage is then used for answer generation.

A. Dataset Construction

To train and evaluate our approach, we used three complementary Arabic QA datasets in our experiments, Arabic SQuAD v2.0 (Best) [19], Arabic Reading Comprehension Dataset (ARCD) [20] and GPT-Generated Ambiguous Data.

1) The Arabic SQuAD v2.0 (Best)

It is a machine-translated and cleaned version of the English SQuAD v2.0 [12]. Some context passages in the dataset exhibit high similarity, which creates natural retrieval ambiguity, making it useful for training ambiguity detection models based on ranking score patterns.

2) Arabic Reading Comprehension Dataset (ARCD)

It is a native Arabic dataset composed of human-authored questions and answers in Modern Standard Arabic, covering a diverse range of topics and linguistic styles. Its organic phrasing and semantic overlap between passages present more realistic cases of ambiguity.

3) GPT-Generated Ambiguous Data

To specifically focus on ambiguity, we used GPT-3.5 Turbo [11] to generate a set of challenging ambiguous questions for newly extracted Wikipedia passages in Arabic. We prompted the model with instructions such as: "Given the passage below, write a question that could be interpreted in multiple ways or could match multiple passages with similar content. Make the question ambiguous by avoiding specific names, dates, or unique details. Use general terms or abstract language that may apply to more than one interpretation of the passage.".

In total, we created about 1,500 Question-Context pairs this way. These synthetic queries were manually reviewed to ensure their linguistic plausibility and ambiguity quality, complementing the naturally ambiguous cases from real datasets. While synthetic data may not fully replicate real-world user queries [26], it helps improve the classifier's generalization by exposing it to diverse patterns of semantic confusion.

The final combined dataset has approximately 10,000 QA pairs. We split this into 80% for training and 20% for testing.
Each question in the dataset is linked to a passage that contains the answer.

Using this dataset allows us to capture different manifestations of ambiguity—synthetic and structural—leading to a more comprehensive evaluation of our detection and interactive clarification framework.

B. Knowledge Graph Construction

We constructed a knowledge graph by grouping documents based on named entities we extracted from each passage using a hybrid approach that combines the strengths of CAMeL Tools [13] and Farasa [14], two state-of-the-art Arabic entity recognition models, to maximize coverage and accuracy in Arabic-language content.

Each document was also associated with an auto-generated title, produced using OpenAI's GPT-3.5 Turbo model [11], which provides a concise and descriptive label that aids in entity disambiguation and user interaction by this prompt: "Read the following passage carefully and generate a clear, concise, and accurate title that summarizes its main idea. The title should cover all important points, highlight any unique details, and avoid ambiguity with similar topics. Make sure the title is specific enough to distinguish this passage from others about related subjects".

The resulting knowledge graph was implemented using Neo4j [17], where entities are represented as nodes and documents as connected entities, allowing efficient retrieval of semantically related content through graph-based traversal. Neo4j's native graph storage and Cypher query language provided a scalable and flexible environment for building and querying the knowledge graph effectively. Nonetheless, scaling such a graph to millions of documents remains challenging, particularly in open-domain or web-scale QA. In future work, we aim to explore more scalable alternatives, such as entity-linking pipelines with dynamic graphs or vector-based knowledge routing [27].

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 21:55:19 UTC from IEEE Xplore. Restrictions apply.

C. Dense Retrieval Models

Our work experiments with dense vector retrievers for multilingual QA.

We use LaBSE (Language-agnostic BERT Sentence Embedding) by Feng et al. [10], a dual-encoder model supporting 100+ languages including Arabic. LaBSE yields high-quality multilingual sentence embeddings and achieved state-of-the-art bi-text retrieval accuracy across many languages.

We also incorporate "Multilingual-E5-Large" model, a dense retrieval model trained using contrastive learning on a large-scale multilingual dataset covering over 100 languages [9]. Based on the Transformer architecture and fine-tuned with a text embedding objective tailored for retrieval tasks. The model has demonstrated strong performance across multiple multilingual information retrieval benchmarks, making it well-suited for cross-lingual retrieval and question answering applications.

Finally, we fine-tuned the LaBSE model on our Arabic QA corpus. Fine-tuning is expected to improve model recall, as seen in other QA systems (e.g., Mozannar et al. achieved higher QA scores by fine-tuning multilingual BERT on Arabic data [7]).

By evaluating LaBSE-base vs. LaBSE-fine-tuned vs. advanced embeddings (E5), we cover a range of retriever strengths in our experiments.

D. Ambiguity Classifier

For the ambiguity detection component, we choose XGBoost [8] due to its efficiency and strong performance on classification tasks.

XGBoost (eXtreme Gradient Boosting) is a tree boosting algorithm known for its scalability and accuracy, having won numerous machine learning competitions. In our context, it allows quick inference to decide if a query is ambiguous, adding negligible latency to the pipeline. Moreover, XGBoost is less prone to overfitting on small feature sets, which suits our use of a limited number of engineered features.

To automatically decide if a query needs clarification, we train the classifier to look at the features of the retrieval results. Intuitively, if the top retrieved documents are all highly relevant and on the same topic, the query was clear. But if the relevance scores are very close for different interpretations, the query might be ambiguous.

We operationalize this intuition with three primary features: score margin, passage diversity and shared entities.

1) Score Margin

The difference between the top-ranked document's relevance score and the second-ranked document's score. An ambiguous query often yields two or more top results with nearly tied scores, since the retriever is unsure which context is more relevant. A small score margin is a strong indicator of ambiguity, whereas a clear query typically has one result with a much higher score than others.

2) Passage Diversity

It is defined as the semantic variance among the top-k retrieved passages. In our setting, lower diversity indicates that the retriever has surfaced several closely related contexts, suggesting potential ambiguity. When many passages appear similarly relevant, the model may be uncertain about which one is the actual target — a hallmark of query ambiguity in our framework.

3) Shared Entities

This feature quantifies the overlap of named entities across the top 5 retrieved passages. The minimal entity overlap indicate an unambiguous result. We extract named entities from each passage, compute the pairwise Jaccard similarity of their entity sets, and average these values across all passage pairs.

In addition to the mentioned features, we extracted several score-based features from the top-k retrieval results to quantify ambiguity. These include other margin scores (e.g., margin_1_3, margin_1_5), which measure the score gaps between the top-ranked passage and lower-ranked ones—smaller margins often indicate uncertainty. We also computed an entropy-like score over the normalized retrieval scores to capture distributional flatness, where higher entropy suggests a lack of a clear winner. Other statistical features include the mean, standard deviation, minimum, and maximum of the top-k scores, offering additional signals about confidence and dispersion. Together, these features provide a nuanced view of retrieval ambiguity based on score dynamics.

Each query's feature vector is fed into the XGBoost classifier, which outputs a probability of the input scores being ambiguous.

We trained this classifier on a labelled set of score lists that were generated using our retrieval models. Each score list is either labelled as ambiguous (label = 0) or unambiguous (label = 1), depending on whether the highest score represents the wrong document or the right one.

While we selected XGBoost for its fast inference time and suitability for real-time QA systems, future improvements may involve neural classifiers that can learn richer semantic patterns, such as fine-tuned transformers or prompt-based LLM scoring [28].

IV. Experiments and Results

A. Measurement Metrics

We integrated our ambiguity detection and clarification module into a retrieval-based QA pipeline and compared its performance to a baseline pipeline without clarification. The primary metric we report is Recall@1 for retrieval, i.e. the percentage of questions for which the top-ranked document contains the correct

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 21:55:19 UTC from IEEE Xplore. Restrictions apply.

answer. This metric directly impacts final QA accuracy, since our reader can only answer correctly if the relevant document is retrieved.

We also measure the classifier's performance by calculating the percentage of correct ambiguity detection cases. Specifically, this involves classifying retrieved scores as ambiguous when the top retrieved document is already incorrect and as not ambiguous when the top retrieved document is already correct.

B. Experimental Setup

1) Retrievers

As mentioned in Section III, we evaluate three dense retrievers using ~2k question-context pairs to show that our approach works broadly (LaBSE – baseline [10], E5-large [9], LaBSE - fine-tuned).

All retrievers return a ranked list of passages for each query. We set (k=20) for initial retrieval for feature calculation.

In our evaluation, we focus primarily on Accuracy@1 as the core metric for assessing retrieval performance. This decision is justified by the nature of our retrieval setup, which only retrieves the top-1 document for each query. In such a case, multiple standard ranking metrics—including Precision@1, Recall@1, F1-score@1, Mean Reciprocal Rank (MRR@1), Mean Average Precision (MAP@1), and Normalized Discounted Cumulative Gain (nDCG@1)—become mathematically equivalent, as they all reduce to a binary judgment of whether the top-ranked document is relevant or not. Therefore, reporting Accuracy@1 alone is sufficient to capture the retrieval model's effectiveness in this strict evaluation setting without loss of generality, as also noted in prior work [21], [22].

2) Ambiguity Classifier

The XGBoost classifier [8] was trained on ~8k score lists labelled as ambiguous vs not by using the retrieval models as described in Section III. We tuned the threshold to achieve high precision on the test set.

We evaluated the ambiguity classifier using standard metrics: accuracy, precision, recall, and F1-score [23]. The goal was to check if the classifier could predict when the top-1 retrieved passage was likely wrong (i.e., ambiguous), using features from the top-k retrieval scores. A correct prediction happens when it flags a query as ambiguous, and the correct passage isn't in the top-1. In our tests, the classifier achieved 92% accuracy, with 85% precision, 71% recall, and an F1-score of 0.78. This demonstrates that the classifier can reliably detect ambiguous queries while minimizing false alarms.

C. Retrieval Performance

TABLE I. RETRIEVAL ACCURACY RESULTS SUMMARY

Retriever Model	***Accuracy@1 (baseline)***	***Accuracy @1 (disambiguates)***
LaBSE (baseline)	76%	87%
E5-large	91%	95%
LaBSE(fine-tuned)	88%	94%

[a.] The retrieval accuracy before and after applying the ambiguity clarification.

As shown above, all three retriever setups see a substantial boost in Accuracy@1 when using our ambiguity mitigation framework. The baseline LaBSE (which had the lowest initial performance at 76%) improved to 87%, a 11-point jump. The stronger E5 model went from 91% to 95% (4-point). Our fine-tuned LaBSE, which started at 88%, reached 94% with clarification (6-point). In relative terms, these are 4–11% increases in the success rate of retrieving the correct answer on top 1. This improvement indicating that many failures were due to ambiguity that is now being resolved. The accuracy gain underscores that certain ambiguous cases cannot be handled by training alone – they truly require disambiguation by the user.

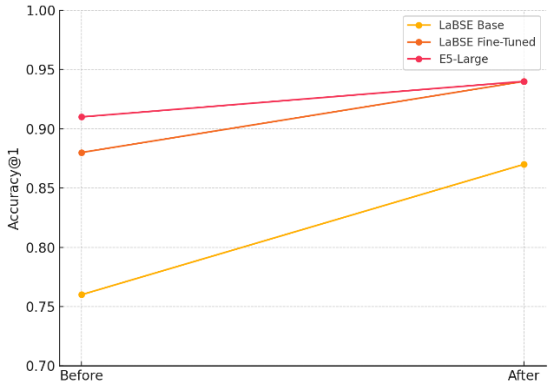

Fig. 2. Impact of ambiguity detection on retrieval accuracy (Accuracy@1) across different retriever models.

As shown in Table II, the ambiguity classifier showed different behaviors depending on the retriever model used. With LaBSE (baseline), the classifier was highly precise (97%), but it missed many ambiguous queries, as indicated by the high false negative rate (~14%). In contrast, using the E5-large retriever helped the classifier better catch ambiguous cases, reducing false negatives to ~3.5%, but it also slightly increased false positives (~3.25%). Overall, the classifier was most balanced with the E5-large model, achieving the highest accuracy (93%) and the best trade-off between catching ambiguity and avoiding false alarms.

This demonstrates that the classifier can reliably detect ambiguous queries while minimizing false alarms. Depending on the retriever used, between 13% and 29% of queries in our evaluation triggered a clarification prompt. For instance, with the LaBSE baseline, the prompt rate reached approximately 28.9% (27.95% true positives + 0.93% false positives), while for E5-large, it

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 21:55:19 UTC from IEEE Xplore. Restrictions apply.

was closer to 13.5% (10.21% + 3.25%). These variations reflect the classifier's operating point and retriever behavior. Across all settings, the false positive rate remained low (typically under 4%), ensuring that the majority—over 70–85%—of user queries proceed uninterrupted, preserving a smooth user experience.

These results highlight the importance of combining a strong retriever with the ambiguity classifier to detect confusion early and avoid retrieval errors.

TABLE II. THE AMBIGUITY CLASSIFIER'S PERFORMANCE

Evaluation Metric	*LaBSE (Baseline)*	*E5-large*	*LaBSE (fine-tuned)*
Accuracy	85%	93%	92%
Precision	97%	76%	86%
Recall	67%	74%	69%
F1-score	79%	75%	77%
True Positives	~27.95%	~10.21%	~13.57%
False Positives	~0.93%	~3.25%	~2.17%
False Negatives	~13.97%	~3.51%	~6.14%
True Negatives	~57.14%	~83.3%	~78.12%

[b.] The ambiguity classifier performance for each retriever model.

V. CONCLUSION

This work tackles ambiguity in RAG-based QA by detecting uncertain retrievals using an XGBoost classifier and prompting users to clarify only when needed. The approach significantly boosts retrieval accuracy with minimal user effort, even in morphologically complex languages like Arabic.

Future work includes extending the method to other languages and domains, exploring neural ambiguity detectors, and conducting user studies to assess interaction impact. We also highlight the need for ethical safeguards—such as privacy protections and bias monitoring—when deploying such interactive QA systems.

VI. REFERENCES

[1] P. Lewis, E. Perez, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.

[2] S. Min, J. Michael, H. Hajishirzi, L. Zettlemoyer, "AmbigQA: Answering Ambiguous Open-domain Questions," EMNLP, 2020.

[3] M. Aliannejadi, H. Zamani, F. Crestani, W. B. Croft, "Asking Clarifying Questions in Open-Domain Information-Seeking Conversations," SIGIR, 2019.

[4] R. Song, Z. Luo, J. Wen, Y. Yu, H. Hon, "Identify Ambiguous Queries in Web Search," WWW, 2007.

[5] S. Cronen-Townsend, W. B. Croft, "Quantifying Query Ambiguity," HLT, 2002.

[6] M. Guo, et al., "Abg-CoQA: Clarifying Ambiguities in Conversational Question Answering," AKBC Workshop, 2021.

[7] H. Mozannar, et al., "Neural Arabic Question Answering," WANLP, 2019.

[8] T. Chen, C. Guestrin, "XGBoost: A Scalable Tree Boosting System," KDD, 2016.

[9] L. Wang, et al., "Text Embeddings by Weakly-Supervised Contrastive Pre-training (E5)," arXiv preprint, arXiv:2212.03533, 2022.

[10] F. Feng, et al., "Language-agnostic BERT Sentence Embedding (LaBSE)," ACL, 2022.

[11] OpenAI, "GPT-3.5 Turbo," OpenAI API Documentation, [Online]. Available: https://platform.openai.com/docs/models/gpt-3-5-turbo. [Accessed: Jul. 15, 2025].

[12] P. Rajpurkar, J. Jia, and P. Liang, "Know What You Don't Know: Unanswerable Questions for SQuAD," Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL), 2018, pp. 784–789. [Online]. Available: https://arxiv.org/abs/1806.03822

[13] K. Obeid, M. Taji, D. Zalmout, Y. O. Albadrashiny, and O. Rambow, "CAMeL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing," Proceedings of the 12th Language Resources and Evaluation Conference (LREC), Marseille, France, 2020, pp. 7022–7032.

[14] W. Abdelali, K. Darwish, N. Durrani, and H. Mubarak, "Farasa: A Fast and Furious Segmenter for Arabic," Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations, San Diego, CA, USA, 2016, pp. 11–16.

[15] F. Radlinski and N. Craswell, "A Theoretical Framework for Conversational Search," Proceedings of the 2017 Conference on Conference Human Information Interaction and Retrieval (CHIIR), 2017, pp. 117–126.

[16] A. Antoun, J. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," Proceedings of the LREC 2020 Workshop on Resources for African Indigenous Languages (RAIL), 2020.

[17] Neo4j, "Neo4j Graph Database," Neo4j Inc., 2024. [Online]. Available: https://neo4j.com. [Accessed: Jul. 15, 2025].

[18] A. Abdul-Mageed, H. Elmadany, and M. Nagoudi, "ARABERT and MARBERT: Deep Bidirectional Transformers for Arabic," Proceedings of the 6th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT), 2021.

[19] M. Makahleh, "Arabic SQuAD v2.0 (Best)," Kaggle, [Online]. Available: https://www.kaggle.com/code/mohammadmakahleh/arabic-squad-v2-0-best. [Accessed: July 14, 2025].

[20] A. Mozannar, R. Siblini, and R. Hajj, "ARCD: Arabic Reading Comprehension Dataset," arXiv preprint arXiv:1906.05394, 2019. [Online]. Available: https://arxiv.org/abs/1906.05394

[21] N. Craswell and B. Mitra, "An Evaluation of Neural Information Retrieval Metrics," in *Proc. 42nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, Paris, France, 2019, pp. 1125–1128.

[22] K. Järvelin and J. Kekäläinen, "IR Evaluation Methods for Retrieving Highly Relevant Documents," in *Proc. 23rd Annu. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, Athens, Greece, 2000, pp. 41–48.

[23] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.

[24] Z. Lyu, C. Yao, and A. Yates, "Zero-Shot Clarification Question Generation via Generative QA Distillation," in Proc. EMNLP, 2023.

[25] D. Khashabi, S. Min, T. Khot, A. Sabharwal, and H. Hajishirzi, "UnifiedQA: Crossing Format Boundaries with a Single QA System," in Proc. EMNLP, 2020.

[26] T. Goyal, D. Khashabi, and P. Clark, "The Dataset Cartography of Question Answering," in Proc. ACL, 2022.

[27] B. Zhao, L. Wang, and K. Lei, "LightKG: Lightweight Graph-Based Entity Routing for Large-Scale Open-Domain QA," in Proc. WWW, 2023.

[28] Y. Wang, Y. Liu, and L. Li, "Prompt-based Ambiguity Detection for Conversational Question Answering," in Proc. NAACL, 2023.

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 21:55:19 UTC from IEEE Xplore. Restrictions apply.